

API

PTP Program



Contents

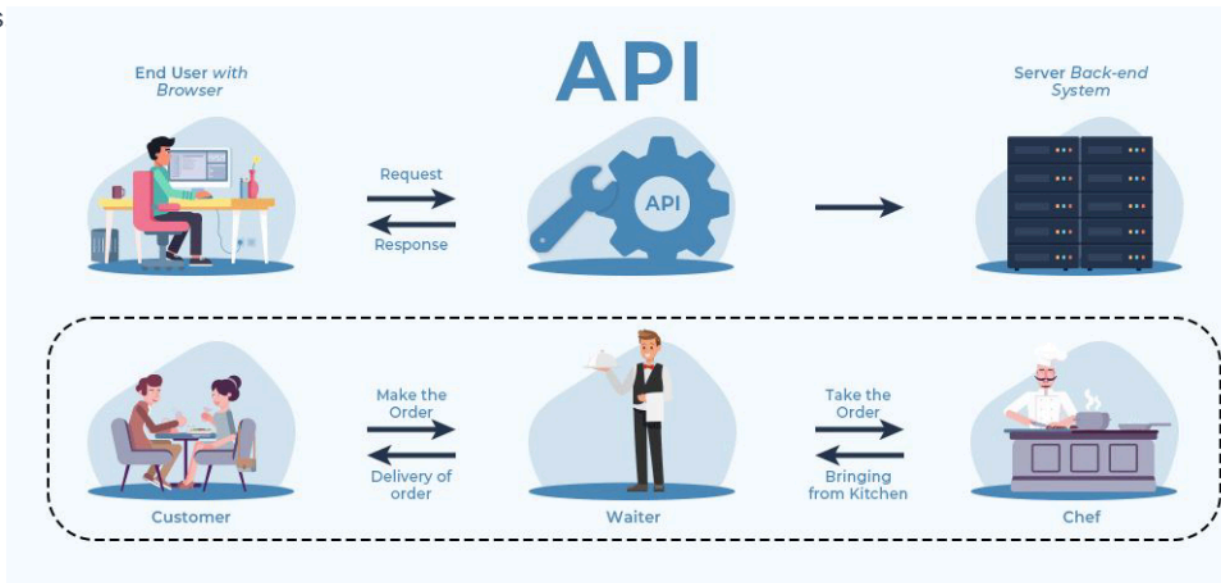


Introduction to API	03
Introduction to REST API	04
Basic HTTP	05
Introduction to JSON	08
Fetching Data from Public API	09
Introduction to FastAPI	10
Introduction to Swagger UI	11
Introduction to Postman	12
Let's Practice!	13
API Headers	14
API Authentication	15
Deploy Your API using Vercel	17

Introduction to API

What is API?

API or Application Programming Interface is a set of rules and protocols that allows different software applications to communicate and interact with each other. It defines the methods and data formats that applications can use to request and exchange information. APIs are commonly used to enable integration between different systems, services, or platforms



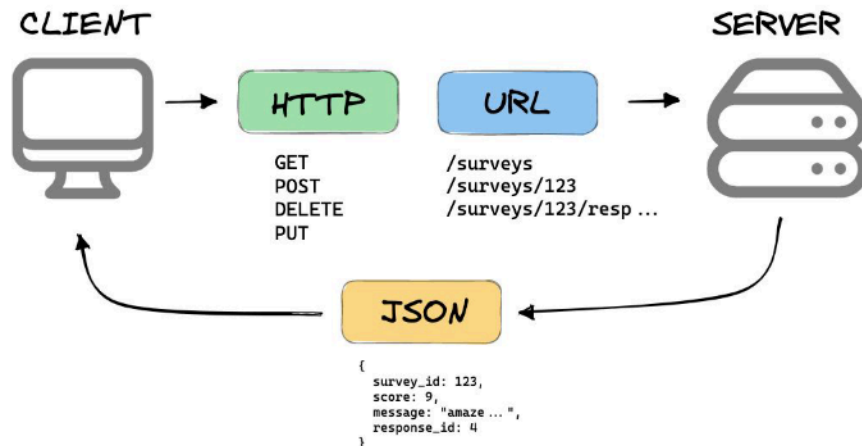
Introduction to REST API

REST API and How It Works

REST stands for Representational State Transfer, and follows the constraints of REST architecture allowing interaction with RESTful web services. It defines a set of functions (GET, PUT, POST, DELETE) that clients use to access server data. The functions used are:

- GET (retrieve a record)
- PUT (update a record)
- POST (create a record)
- DELETE (delete the record)

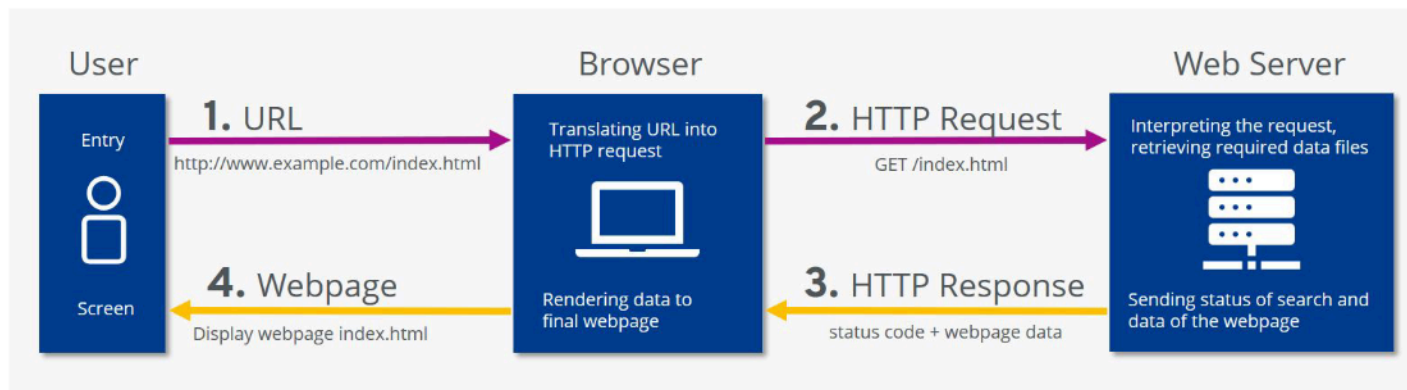
Its main feature is that REST API is statelessness, i.e., the servers do not save clients' data between requests.



Basic HTTP

Introduction to HTTP and Communication Process

HTTP (Hypertext Transfer Protocol) is a protocol that allows communication between clients and servers over the internet. It specifies the structure and semantics of messages exchanged between the client (such as a web browser) and the server (such as a web server).



Basic HTTP

HTTP Response Status Codes

HTTP response status codes are three-digit numbers returned by a server to indicate the status of a client's HTTP request. They provide information about whether the request was successful, encountered an error, or requires further action. Here are some commonly used HTTP response status codes:

1. 1xx Informational:
 - **100 Continue:** The server has received the initial part of the request and expects the client to continue with the rest.
2. 2xx Success:
 - **200 OK:** The request was successful, and the server returns the requested resource as the response.
 - **201 Created:** The request was successful, and a new resource has been created as a result.
 - **204 No Content:** The request was successful, but there is no content to return in the response.
3. 3xx Redirection:
 - **301 Moved Permanently:** The requested resource has been permanently moved to a new location.
 - **302 Found/Redirect:** The requested resource is temporarily available at a different location.
 - **304 Not Modified:** The client's cached version of the requested resource is up to date, and no further action is needed.

Basic HTTP

HTTP Response Status Codes

4. 4xx Client Errors:

- **400 Bad Request:** The server cannot understand the request due to malformed syntax or invalid parameters.
- **401 Unauthorized:** The request requires authentication, and the client must provide valid credentials.
- **403 Forbidden:** The client does not have permission to access the requested resource.
- **404 Not Found:** The requested resource could not be found on the server.

5. 5xx Server Errors:

- **500 Internal Server Error:** The server encountered an unexpected condition that prevented it from fulfilling the request.
- **502 Bad Gateway:** The server acting as a gateway received an invalid response from an upstream server.
- **503 Service Unavailable:** The server is temporarily unable to handle the request due to maintenance or high load.
- **504 Gateway Timeout:** The server acting as a gateway did not receive a timely response from an upstream server.

Introduction to JSON

JSON

JSON (JavaScript Object Notation) is a lightweight data interchange format commonly used for storing and exchanging structured data between systems. It is human-readable and easy for both humans and machines to parse and generate. JSON data is represented in key-value pairs and organized in a hierarchical structure like **dictionary** in Python. Typically, this format will be saved into a **.json** file.

```
{
  "name": "John Doe",
  "age": 30,
  "isStudent": false,
  "address": {
    "street": "123 Main St",
    "city": "New York"
  },
  "hobbies": ["reading", "playing guitar", "hiking"]
}
```


Introduction to

Fetching Data from API

```
import requests

# API endpoint URL
url = 'https://api.chucknorris.io/jokes/random'

# Send GET request to the API
response = requests.get(url)

# Check if the request was successful (status code 200)
if response.status_code == 200:
    # Parse the response as JSON
    data = response.json()

    # Extract the joke from the response
    joke = data['value']

    # Print the joke
    print(joke)
else:
    # Request failed, handle the error
    print(f'Request failed with status code: {response.status_code}')
```

Fetching data from an API means that you receive data from accessing the API. To do that, you need to determine the endpoint URL of the API you want to fetch data from. This URL represents the location where the API exposes its resources.

Beside image is an example of how to fetch data from an API using **requests** library. In this example, we want to fetch random joke from chuck norris website.

Introduction to

FastAPI

FastAPI is a modern, high-performance web framework for building APIs with Python. It is built on top of the Starlette framework and takes advantage of Python 3.7+ type hints to provide efficient and type-safe APIs.



```
from fastapi import FastAPI

# Create an instance of the FastAPI class
app = FastAPI()

# Define a route (endpoint) for the root path "/"
@app.get("/")
def read_root():
    return {"Hello": "World"}

# Define another route with a path parameter
@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}
```

Introduction to

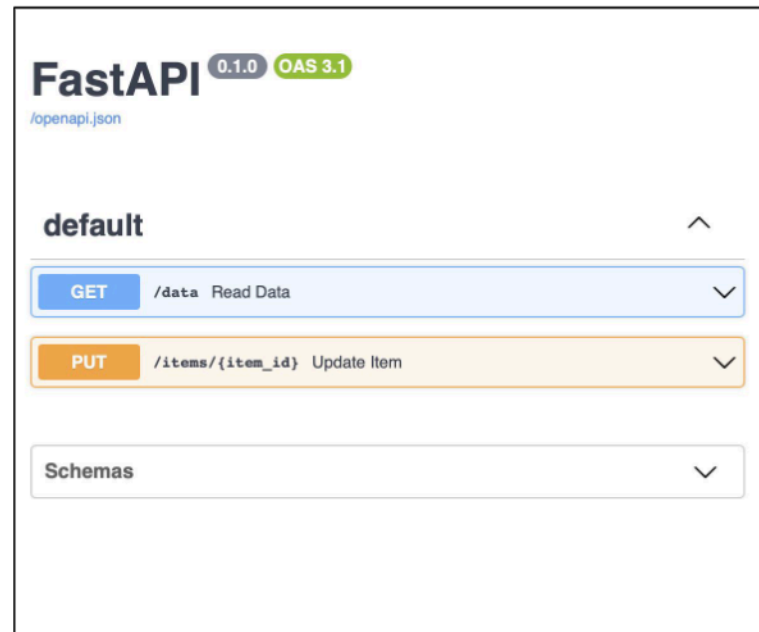
Swagger UI

Swagger UI is a tool that used to generate interactive API documentation that lets your users try out the API calls directly in the browser.

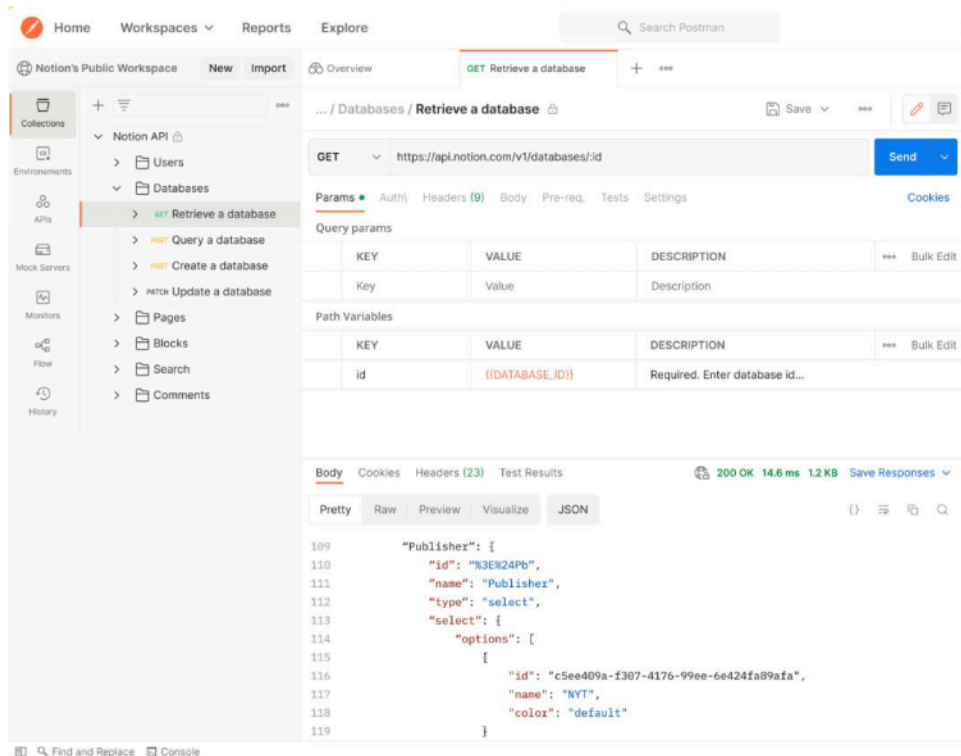
Since you use FastAPI to build the APIs, so you don't need install anything to access the Swagger UI. You only need to access the 'docs' of your API.

To access it, use the url like this:

- <https://localhost:8000/docs>, or
- <https://yourapiurl.com/api/docs>



Introduction to Postman



Postman is a platform for API development. It is used by developers to design, test, and document APIs more efficiently.

Postman provides a user-friendly interface that allows users to make API requests, test API functionality, view responses, and manage different aspects of API development.

To use Postman, you can either download the Postman application for your operating system or use the web version available at <https://www.postman.com/>.

Introduction to

API Headers

API headers are additional pieces of information that can be included in an HTTP request or response. They provide metadata and instructions to the client or server to handle the request or response appropriately.

Headers are key-value pairs, where the key represents the name of the header, and the value contains the associated data.

Introduction to

API Authentication

API authentication is the process of verifying the identity of clients and granting them access to protected resources or functionalities within an API. It ensures that only authorized users or applications can interact with the API and perform specific actions.

API authentication typically involves the following components and mechanisms:

1. **API Keys:** Unique identifiers issued to clients for authentication and tracking API usage.
2. **Tokens:** Cryptographic strings issued to authenticated clients for subsequent requests and identity verification.
3. **OAuth:** Authentication and authorization protocol for granting access to protected resources without sharing credentials.
4. **Basic Authentication:** Simple authentication scheme where clients include username and password in request headers.

Introduction to

API Authentication

5. **Bearer Tokens:** Authentication tokens included in request headers to authenticate clients accessing protected resources.
6. **Multi-Factor Authentication (MFA):** Additional verification factors (e.g., temporary codes) required alongside credentials for enhanced security.
7. **API Key Rotation and Expiration:** Periodically renewing API keys and tokens to enhance security and prevent unauthorized access.
8. **Rate Limiting:** Controlling the number of requests a client can make within a specific time period to prevent abuse and maintain API performance.

Deploy Your API using **Vercel**

Vercel is a cloud platform designed to empower developers with a seamless way to deploy and host web applications, enabling them to focus on building exceptional user experiences. It's an all-in-one solution that supports both frontend and serverless backend deployment, perfect for modern web development.

Vercel is very easy to use. You only need to sign up and manage your app with your Github. You may look at the brief tutorial to deploy your API.

To access the app, enter this link: <https://vercel.com>

Let's

Practice!

We will perform to build a simple API using FastAPI. You will need Visual Studio Code. Check the Google Colab on the last slide to see the tutorial and instructions.

External References

Basic Syntax



[Visit Here](#)

Deployment



[Visit Here](#)



Hacktiv8id



Hacktiv8id



Hacktiv8



Hacktiv8 Indonesia